
DESARROLLO E IMPLEMENTACIÓN DEL PROYECTO DE TÍTULO

SecureQR

Asignatura: Proyecto de Título – TIHI84

Sección: D-IINF-N8-P1-C3

Académico Guía: Emilio Soto Rojas

Integrantes del equipo: Samantha Caro – Rode Palma

Fecha de entrega: 05/12/2025

Contents

Contents

DESARROLLO E IMPLEMENTACIÓN	1
DEL PROYECTO DE TÍTULO	1
SecureQR	1
1 Introducción	4
2 Detalle de las Tecnologías por implementar	5
2.1 Análisis cualitativo/cuantitativo de las tecnologías que serán implementadas	5
2.2 Herramientas, aplicaciones, lenguajes y componentes que serán implementados	6
3 Detalle de la Arquitectura para implementar	7
3.1 Diagrama BPMN	7
3.2 Diagramas de caso de uso	7
3.3 Diagrama de componentes	8
3.4 Modelo de datos	8
3.5 Topología de comunicaciones	9
3.6 Diagrama de Infraestructura	10
3.7 Diagrama de Arquitectura	11
4 Implementación de los KPI Y SLA	12
4.1 Descripción de KPI	12
4.2 Descripción de SLA	12
5 Plan de Pruebas y Aseguramiento de Calidad	12
5.1 Plan de Pruebas	12
5.2 Normas y Estándares	14
6 Plan de Implementación	15
6.1 Plan de Implementación y despliegue (Google Play)	15
6.2 Gestión de Disponibilidad	15
6.3 Gestión de Continuidad	16
6.4 Plan de Mantenimiento	16
7 Sostenibilidad y Modelo de Negocio	17

7.1	Objetivo de negocio	17
7.2	Alternativas de modelo de negocio	17
7.3	Estrategia de protección de datos	18
8	Plan de Presupuesto	18
9	Ajustes del Cronograma	19
9.1	Revisión de Carta Gantt o desarrollo de fases	19
10	Conclusiones:	20
11	Referencias Bibliográficas	21
12	Anexos	21

1 Introducción

El proyecto SecureQR nace como una respuesta directa y especializada a la amenaza emergente y creciente del Quishing (phishing perpetrado mediante códigos QR). En el contexto digital actual, la adopción de códigos QR se ha masificado, integrándose en sistemas de pago, menús de restaurantes, terminales de transporte y sistemas de autenticación. Si bien esta adopción ha agilizado las interacciones, también ha abierto un nuevo vector de ataque que elude las defensas tradicionales. A diferencia del phishing por correo electrónico, el Quishing utiliza una imagen (el QR) para ocultar un enlace malicioso, evadiendo los filtros de spam basados en texto y explotando la confianza inherente que los usuarios depositan en la inmediatez de sus dispositivos móviles.

Frente a este panorama, se propone el desarrollo de SecureQR: una aplicación móvil nativa para la plataforma Android, diseñada bajo una filosofía de gratuidad y privacidad absoluta (privacy-first). La arquitectura de la solución es fundamentalmente de cliente-ligero, operando sin un backend o servidor propio, lo que garantiza el completo anonimato del usuario y elimina el riesgo de brechas de datos centralizadas. El stack tecnológico seleccionado se basa en Kotlin como lenguaje principal, la librería de escaneo de código abierto ZXing para la decodificación, y la API Google Safe Browsing como servicio externo para la validación de amenazas en tiempo real.

La propuesta de valor de SecureQR se articula sobre un enfoque dual: tecnológico y educativo. El pilar tecnológico no se limita a escanear, sino que interrumpe el flujo de "clic inmediato" al implementar una "zona de espera" (sandbox visual). Esta zona muestra al usuario la URL completa y decodificada antes de que el navegador o cualquier otra aplicación la procese, combinando esta inspección visual con la validación técnica de Google. De forma paralela, el pilar educativo busca mitigar el riesgo a largo plazo, incorporando un módulo de concienciación proactiva con buenas prácticas en ciberseguridad, transformando la herramienta de un simple escudo reactivo a una plataforma de fortalecimiento de la higiene digital del usuario.

El objetivo estratégico y de negocio principal es, por tanto, posicionar a SecureQR como una herramienta de ciberseguridad ciudadana; un bien público digital que sea gratuito, confiable y educativo. El activo más valioso del proyecto no es el código fuente, sino la confianza del usuario. Esta premisa dicta todas las decisiones de arquitectura, desde la ausencia de un backend hasta el modelo de sostenibilidad, adhiriéndose estrictamente a los principios de "no exigir datos personales" y "evitar cualquier tipo de rastreo".

2 Detalle de las Tecnologías por implementar

2.1 Análisis cualitativo/cuantitativo de las tecnologías que serán implementadas.

La selección del stack tecnológico se fundamentó en un análisis riguroso que priorizó la seguridad, el rendimiento nativo y el control sobre el hardware por encima de la facilidad de desarrollo multiplataforma. Plataforma: se optó por Android nativo debido a su amplia adopción global (más del 70% del mercado móvil) y compatibilidad con librerías especializadas de escaneo y validación de seguridad.

- Otras opciones, como Flutter o React Native, fueron descartadas por menor control de hardware y dependencia de capas intermedias.
- Lenguaje de programación: Kotlin fue elegido por su concisión, interoperabilidad con Java y soporte oficial de Google. Permite desarrollar aplicaciones más seguras (null safety) y con menos líneas de código que Java, sin sacrificar rendimiento
- Motor de lectura QR: la librería ZXing (Zebra Crossing), ampliamente usada en Android, ofrece procesamiento eficiente de códigos 1D y 2D, además de código abierto, lo que permite su integración y personalización.
- Gestor de base de datos: SQLite, integrado de forma nativa en Android, proporciona una solución ligera y transaccional para almacenar datos locales (historial de escaneos, configuraciones, registros de seguridad).
- Servicios externos: Google Safe Browsing API, para validar enlaces en tiempo real contra bases de datos de phishing y malware, garantizando actualizaciones automáticas y detección confiable.

Tecnología / Herramienta	Rendimiento	Seguridad	Compatibilidad	Costo	Selección
Android Studio + Kotlin	Alto	Alto	Amplia	Gratuito	Si
Flutter / React Native	Medio	Medio	Alta	Gratuito	No
ZXing	Alto	Alto	Amplia	Gratuito	Si
SQLite	Alto	Medio	Amplia	Gratuito	Si
Google Safe Browsing API	Alto	Muy alto	Global	Gratuito (con Limitaciones)	Si

2.2 Herramientas, aplicaciones, lenguajes y componentes que serán implementados

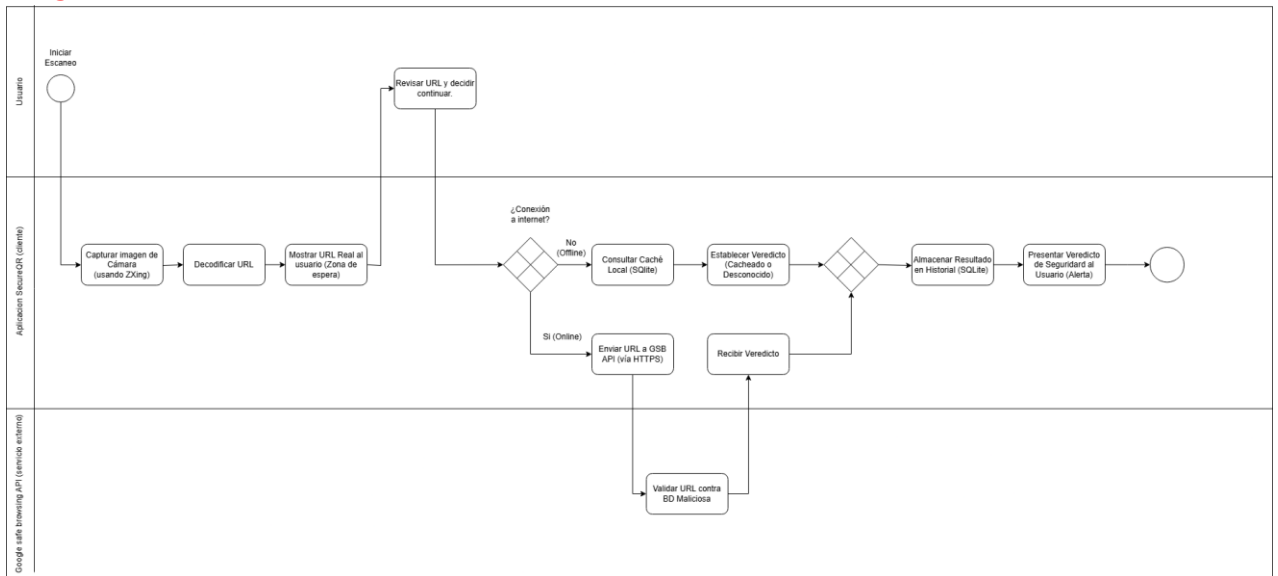
Cada componente ha sido seleccionado para reforzar la arquitectura de cliente-ligero, privacidad y seguridad:

- **Lenguaje (Kotlin):** Es el lenguaje preferido y oficial para el desarrollo moderno de Android. Más allá de su concisión y soporte para programación reactiva (Coroutines), su característica de Null Safety (seguridad contra nulos) es un pilar de seguridad. Esta característica elimina en tiempo de compilación las NullPointerExceptions, una de las vulnerabilidades y causas de fallos (crashes) más comunes en aplicaciones Java, resultando en una base de código intrínsecamente más segura y robusta.
- **Motor de Escaneo (ZXing - "Zebra Crossing"):** Es la librería de código abierto estándar de la industria para el procesamiento de códigos 1D/2D. Su naturaleza open-source es una ventaja de seguridad crítica: permite la auditoría completa del código, asegura que no existan rastreadores ocultos y garantiza que la librería solo realiza la tarea de decodificación, sin transmisiones de datos no autorizadas.
- **Persistencia Local (SQLite con wrapper Room):** Dada la arquitectura cliente-ligera, toda la persistencia de datos ocurre en el dispositivo. SQLite es el motor de base de datos serverless y transaccional integrado nativamente en Android. Se gestionará a través de Room, una capa de abstracción de Android Jetpack que facilita la creación y gestión de la base de datos, reduce el código boilerplate y añade comprobaciones en tiempo de compilación de las consultas SQL, previniendo errores de inyección SQL locales. Almacenará el historial y las configuraciones de forma anónima.
- **Validación de Amenazas (Google Safe Browsing API v4):** Este servicio proporciona acceso a la lista de sitios no seguros de Google, actualizada constantemente, que incluye amenazas de phishing y malware. Su uso permite a SecureQR externalizar la inteligencia de amenazas a un líder global, garantizando una alta tasa de detección (KPI > 95%) sin la necesidad de mantener, financiar o proteger una infraestructura de backend propia.
- **Seguridad de Comunicación (TLS/SSL):** Todas las comunicaciones entre la aplicación y servicios externos (exclusivamente la GSB API) serán cifradas obligatoriamente mediante HTTPS, forzando el uso de protocolos modernos como TLS 1.2+ (Transport Layer Security). Esto garantiza la Autenticación del servidor (asegurando que la app habla con Google), la Confidencialidad de los datos (la URL escaneada no puede ser leída por terceros en la red) y la Integridad de la transacción (la respuesta de "seguro" o "peligroso" no puede ser alterada en tránsito). Esto mitiga activamente los ataques Man-in-the-Middle (MitM).
- **Protección de Datos y Cumplimiento (GDPR / Ley 19.628):** La arquitectura implementa los principios de Privacy by Design y Privacy by Default. Al no recolectar PII (Información Personal Identificable) y no tener un backend de usuario, la postura de cumplimiento ante normativas como GDPR y la Ley 19.628 de Chile se simplifica drásticamente. El único dato transmitido (la URL) no está asociado a una identidad. La Política de Privacidad será transparente, declarando explícitamente que las URLs se envían a Google para validación y que todo el historial es local y anónimo.

3 Detalle de la Arquitectura para implementar

3.1 Diagrama

BPMN



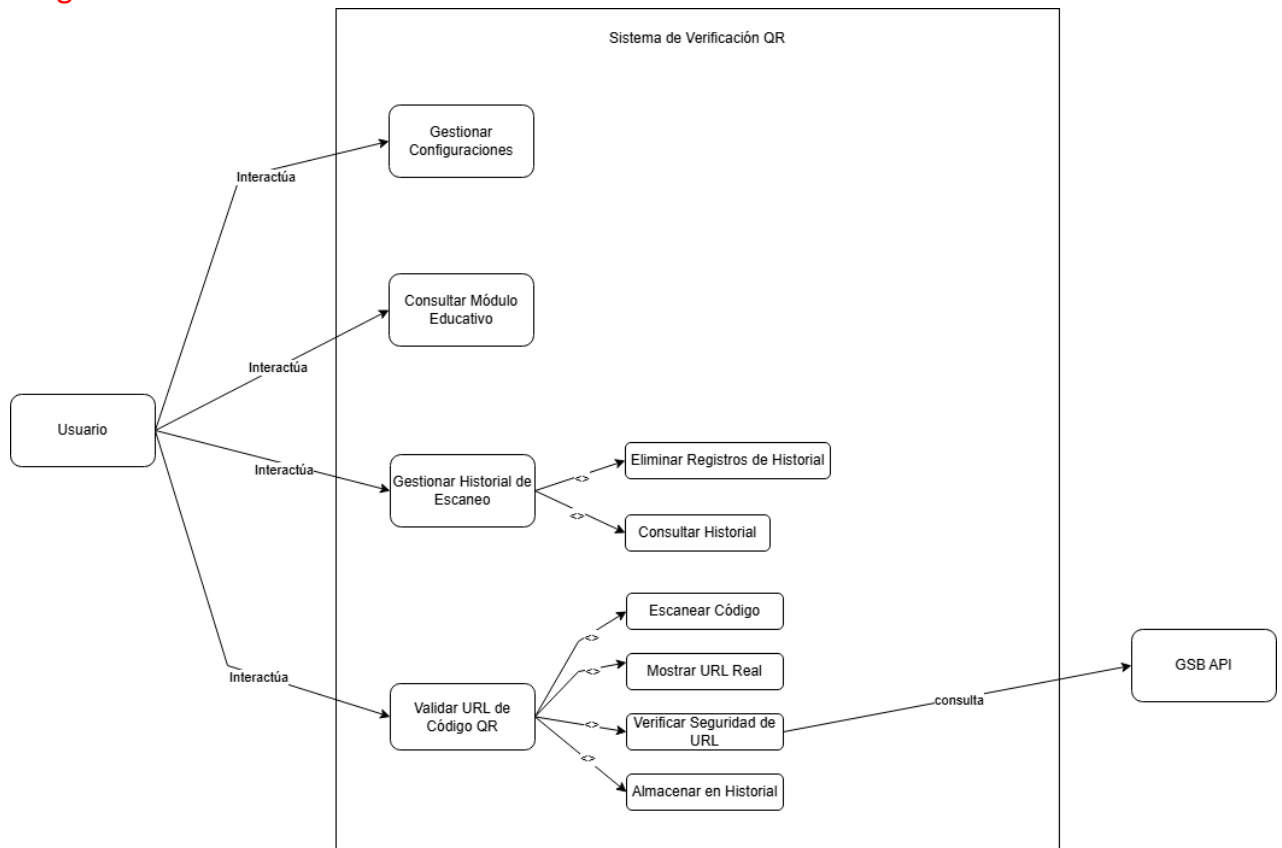
3.2 Diagramas

de

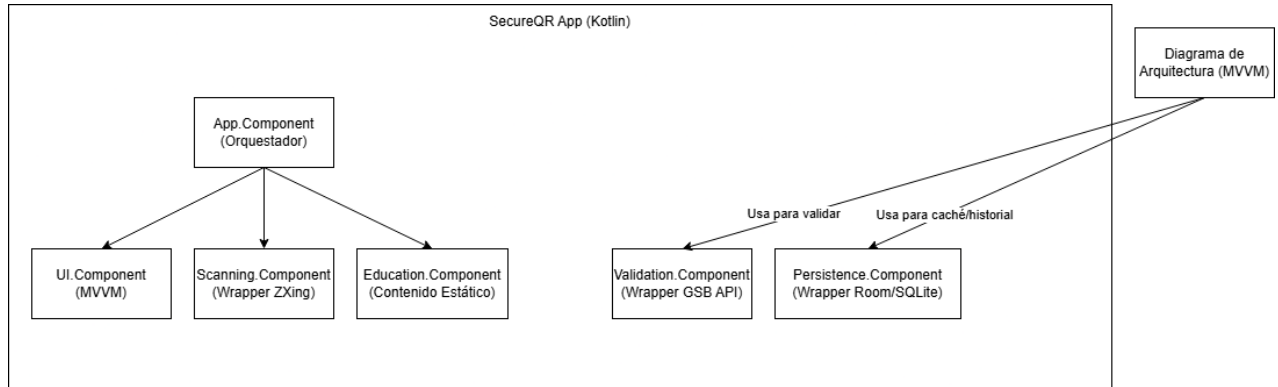
caso

de

uso



3.3 Diagrama de componentes



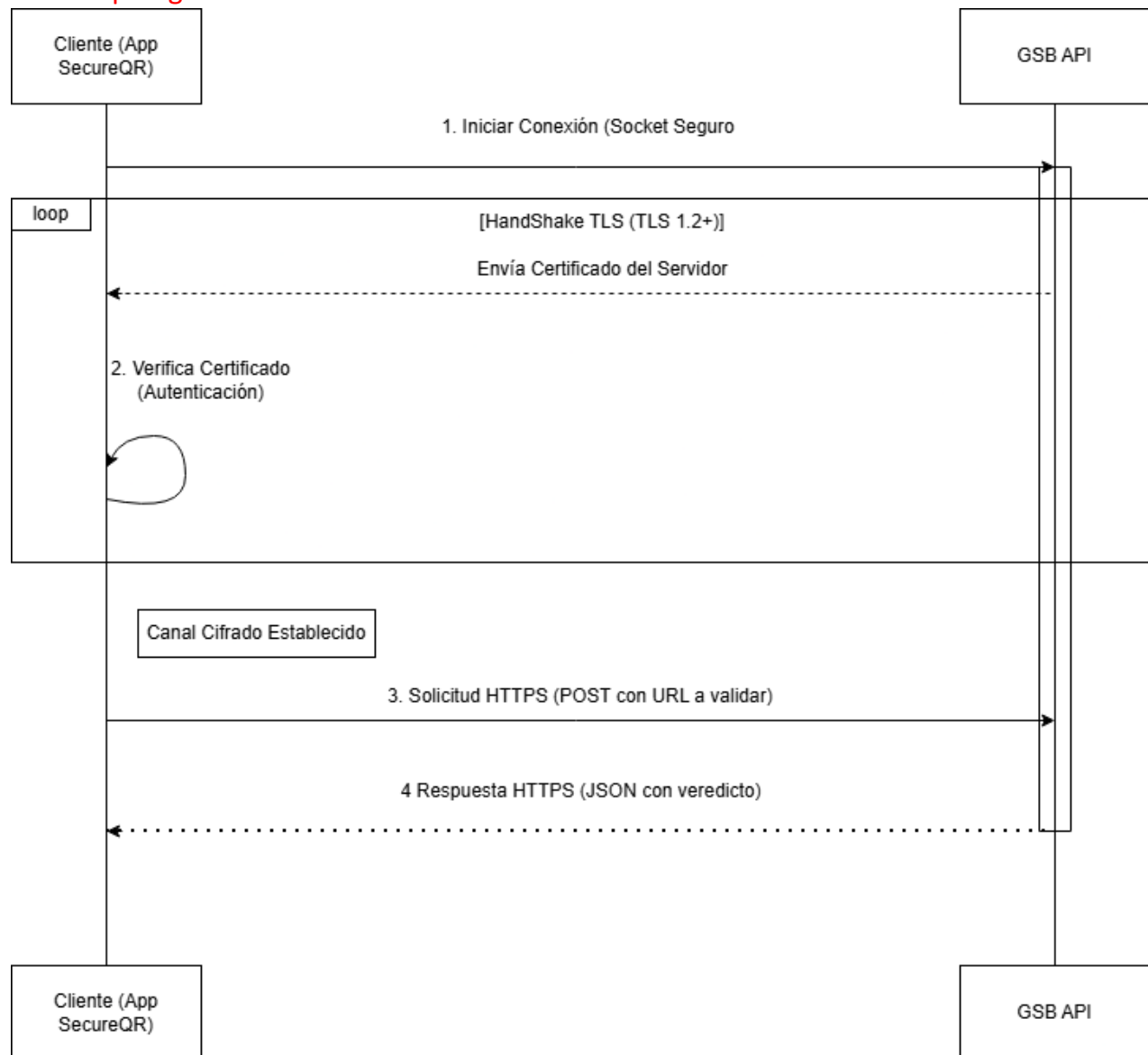
3.4 Modelo de datos

HistorialEscaneos
INTEGER id PK AUTOINCREMENT
TEXT scanned_url NOT NULL
TEXT url_hash NOT NULL
INTEGER security_status NOT NULL (0=Seguro, 1= Malicioso, 2=Desc)
INTEGER scan_timestamp NOT NULL
INTEGER api_validation_timestamp Timestamp

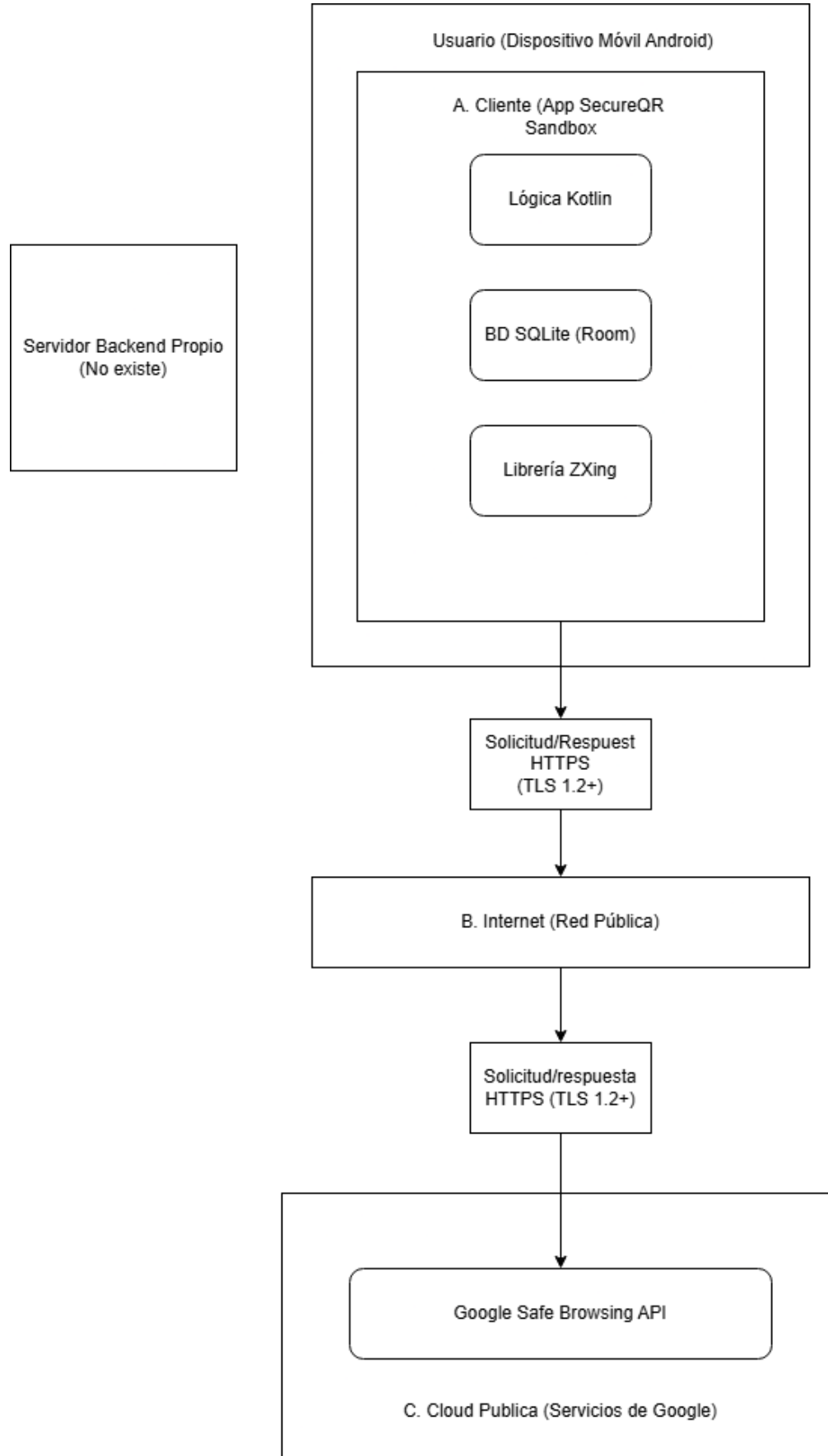
AppSettings
TEXT config_key PK
TEXT config_value NOT NULL

Educational Content
INTEGER article_id PK
TEXT title NOT NULL
TEXT content_body NOT NULL
TEXT category

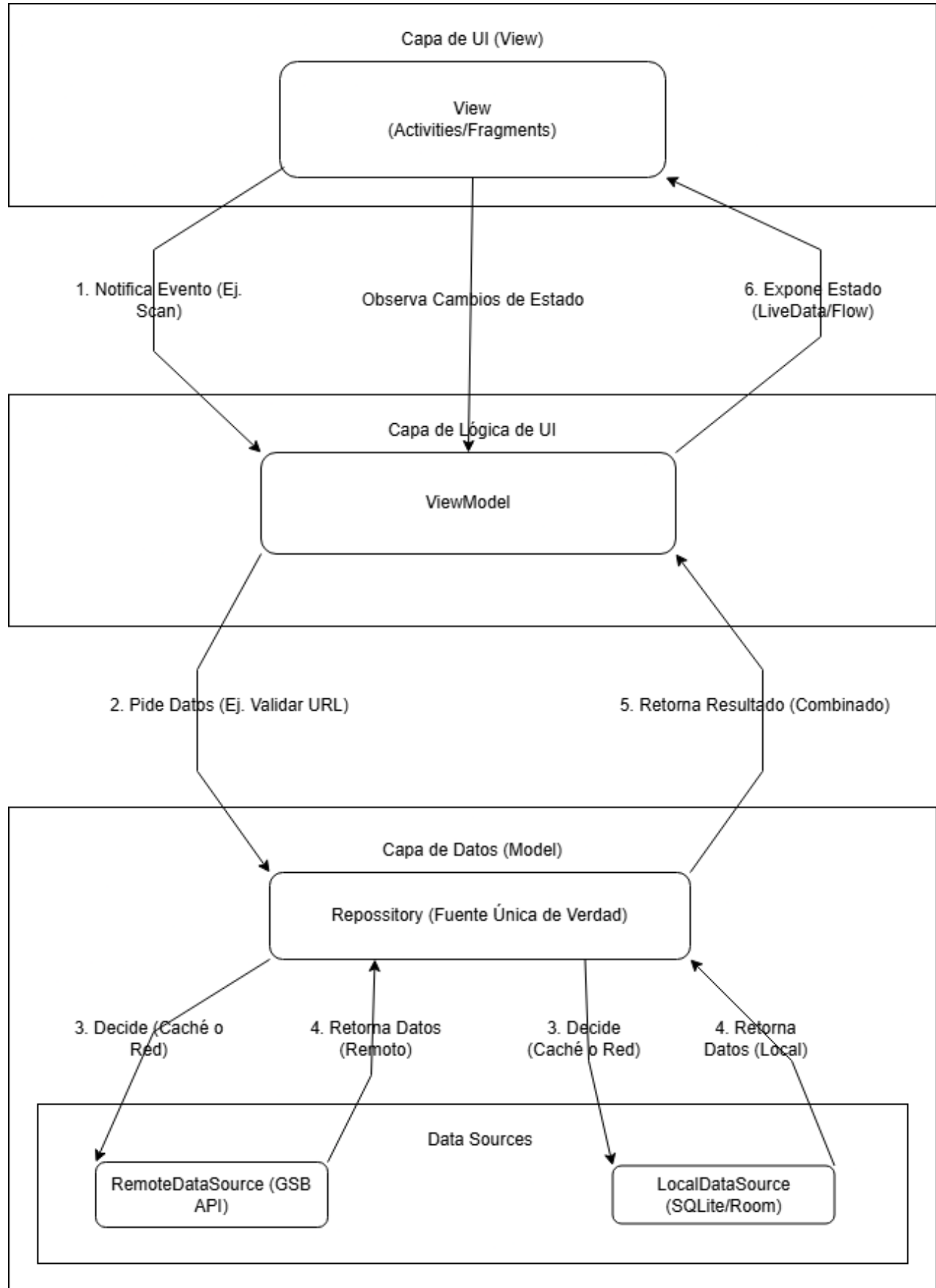
3.5 Topología de comunicaciones



3.6 Diagrama de Infraestructura



3.7 Diagrama de Arquitectura



4 Implementación de los KPI Y SLA

4.1 Descripción de KPI

El análisis del BPMN (Sección 3.1) reveló la ambigüedad del KPI "Tiempo medio de escaneo". Para que los indicadores sean accionables y midan el rendimiento real de la solución (y no del usuario), se refinan los KPIs:

Indicador	Descripción	Meta
Tiempo medio de escaneo	Desde el inicio hasta validación	< 3 Segundos
Tasa de detección	URLs maliciosas correctamente clasificadas	≥ 95%
Falsos positivos	URLs seguras marcadas como inseguras	< 2%
Disponibilidad	Porcentaje operativo del sistema	99.9%
Precisión global	Validaciones correctas sobre total	≥ 95%

4.2 Descripción de SLA

Los Acuerdos de Nivel de Servicio (SLA) se establecen para los procesos críticos 1, reconociendo las dependencias externas:

- SLA 1: Disponibilidad del Servicio de Validación (Proceso Crítico Externo): Se establece una meta de 99.9%. Es fundamental entender que este es un SLA heredado de la Google Safe Browsing API. La responsabilidad del proyecto SecureQR no es garantizar este 99.9%, sino gestionar la falla cuando ocurra (ver 6.2 Gestión de Disponibilidad).
- SLA 2: Disponibilidad de Funciones Locales (Procesos Críticos Internos): Las funciones críticas que no dependen de la red (Escaneo para mostrar URL, consulta de Historial, acceso al Módulo Educativo) tendrán un SLA de 99.99%. Este SLA sí es controlable por el equipo de desarrollo y depende de la calidad del código (ausencia de crashes).
- SLA 3: Tiempos de Respuesta (Proceso Crítico Total): El tiempo total del proceso técnico (TDC + TVR) no debe exceder los 3 segundos 1 bajo condiciones de red óptimas (Wi-Fi/5G).

5 Plan de Pruebas y Aseguramiento de Calidad

5.1 Plan de Pruebas

El plan se expande para incluir los tipos de pruebas solicitados (alfa, beta, estrés, de humo, unitarias, integrales), definiendo qué se probará, por qué y cuándo.

- Pruebas Unitarias (Unit Testing):
 - Qué: Pruebas automatizadas de clases individuales en aislamiento.
 - Por qué: Para validar la lógica del ViewModel (ej. formateo de estado) y del Repository (ej. lógica de decisión de caché) sin depender de la UI o la red.
 - Oportunidad: Continuamente durante el desarrollo (Sprints 1-3).
- Pruebas de Integración (Integration Testing):
 - Qué: Pruebas (a menudo automatizadas) que verifican la interacción entre componentes.
 - Por qué: Para asegurar que el Repository interactúa correctamente con el LocalDataSource (Room/SQLite) y que el ViewModel recibe los datos correctos del Repository.

- Oportunidad: Al final de los Sprints 2 y 3.
- Pruebas de Humo (Smoke Testing):
 - Qué: Un subconjunto pequeño y rápido de pruebas manuales o automatizadas sobre una build nueva.
 - Por qué: Para verificar que las funcionalidades críticas no están rotas (ej. ¿La app abre? ¿La cámara inicia? ¿Escanea un QR?). Si falla, la build se rechaza para QA.
 - Oportunidad: Al inicio de cada sesión de QA (Fase 5).
- Pruebas Funcionales (QA Manual):
 - Qué: Validación manual de todos los casos de uso contra los requisitos.
 - **Casos de Prueba Específicos:**
 1. Happy Path: Escanear QR de "google.com". Resultado: Seguro (Verde).
 2. Ataque: Escanear QR de prueba de phishing (ej. EICAR). Resultado: Alerta Maliciosa (Rojo).
 3. UI: Verificar que una URL muy larga se muestra correctamente (no truncada) en la "zona de espera".
 4. Historial: Verificar que los Casos 1 y 2 aparecen correctamente en el historial SQLite.
 5. Offline (Caché Vacía): Activar modo avión. Escanear QR nuevo. Resultado: "Validación no disponible" (Gris).
 6. Offline (Caché Llena): Escanear "google.com" (online). Activar modo avión. Escanear mismo QR. Resultado: Mostrar veredicto cacheado "Seguro" (Verde).
 - Oportunidad: Durante toda la Fase 5 (QA y Refinamiento).
- Pruebas de Estrés y Rendimiento (Stress Testing):
 - Qué: Pruebas para medir el comportamiento bajo condiciones adversas.
 - Casos: Medir KPIs TDC y TVR bajo redes lentas (3G); escanear 50 QR seguidos rápidamente (medir consumo de memoria y batería); verificar la app durante llamadas entrantes.
 - Oportunidad: Fase 5.
- Pruebas de Seguridad (Manual):
 - Qué: Intentos de vulnerar la aplicación.
 - Casos: Interceptar el tráfico de la app (ej. Burp Suite) y verificar que es ilegible (HTTPS) y que la validación falla si el certificado es falso (MitM); extraer el archivo de la BD SQLite del dispositivo (con root) y verificar que no hay PII almacenado.
 - Oportunidad: Fase 5.
- Pruebas de Aceptación (Alfa/Beta):
 - Pruebas Alfa: Pruebas internas por el equipo en dispositivos físicos (no emuladores).
 - Pruebas Beta: Lanzamiento controlado a un grupo pequeño de usuarios externos (vía Google Play Console) para obtener feedback del mundo real.
 - Oportunidad: Final de la Fase 5 / Inicio de Fase 6.
- Pruebas de Compatibilidad:
 - Qué: Validar funcionamiento en diferentes hardware y software.
 - Casos: Ejecutar las pruebas funcionales en Android 8.0, Android 10 y Android 14; probar en dispositivos físicos (Samsung, Google Pixel, Xiaomi) para validar el acceso a la cámara.
 - Oportunidad: Fase 5.

5.2 Normas y Estándares

El proyecto se adherirá a los siguientes estándares 1 para garantizar un producto de alta calidad y seguro:

- OWASP Mobile Top 10 (2024): Se utilizará como guía principal para la prevención de vulnerabilidades. La arquitectura mitiga proactivamente riesgos clave:
 - M5: Insecure Communication: Mitigado por la imposición de TLS 1.2 para toda comunicación.
 - M6: Inadequate Privacy Controls: Mitigado por la arquitectura anónima y sin backend.
 - M8: Security Misconfiguration: Mitigado al no tener un backend que configurar y al seguir las guías de Android para el sandbox de la app.
- Material Design 3 (MD3) 1: Estándar oficial de Google para la consistencia visual y de usabilidad en Android. Se utilizarán componentes estándar de la librería Jetpack para las alertas, tarjetas de historial y navegación, asegurando una experiencia de usuario intuitiva y accesible.
- ISO/IEC 25010 (Calidad de Producto Software): El plan de pruebas está diseñado para evaluar características clave de este estándar:
 - Adecuación Funcional: (Compleitud, Corrección) - Validado por Pruebas Funcionales.
 - Eficiencia de Desempeño: (Comportamiento temporal) - Validado por Pruebas de Rendimiento (KPIs TDC/TVR).
 - Usabilidad: (Aprendizabilidad, Operabilidad) - Validado por Pruebas Beta y adherencia a Material Design.
 - Fiabilidad: (Tolerancia a fallos, Capacidad de recuperación) - Validado por Pruebas de Estrés y Casos Offline (Caché).

6 Plan de Implementación

6.1 Plan de Implementación y despliegue (Google Play).

El plan sigue una metodología Agile y culmina con el despliegue en Google Play Store:

1. **Fase 1-5: Desarrollo y QA (Ver Cronograma 7.1):** Desarrollo de la aplicación siguiendo el plan de Sprints.
2. **Fase 6: Preparación para Google Play:**
 - Crear cuenta de Desarrollador de Google Play (pago único).
 - Completar la Ficha de Play Store:
 - Nombre, descripción breve, descripción completa (optimizadas para ASO).
 - Recursos gráficos (ícono, *feature graphic*, capturas de pantalla).
 - Completar la Política de Privacidad: Redactar y alojar la política que declara el no uso de PII y el envío de URLs a Google.
 - Completar Cuestionario de Clasificación de Contenido (IARC).
 - Configurar Precio (Gratuito) y Distribución (Global).
3. **Fase 7: Lanzamiento y Pruebas Beta:**
 - Generar el **Android App Bundle (AAB)** firmado (no APK).
 - Subir el AAB al "Canal de Pruebas Cerrado" (Beta) en Play Console.
 - Distribuir a un grupo de *beta testers* (internos y externos).
 - Recopilar *feedback* y reportes de errores (vía Play Console y canal de *feedback*).
4. **Fase 8: Lanzamiento a Producción:**
 - Promover la *build* Beta estable al canal de Producción (Lanzamiento por etapas: 10%, 50%, 100%).
 - Monitorear analíticas de *crashes* y *feedback* inicial.

6.2 Gestión de Disponibilidad

La gestión de disponibilidad se enfoca en la resiliencia de la app ante fallos de red, implementando un patrón de circuit breaker y caché local:

- API de Alta Disponibilidad: Se depende del SLA robusto de Google Safe Browsing.
- Gestión de Fallo de Red (Offline): Si la GSB API no está accesible (por falta de conexión o fallo del servicio), la app no falla. El Repository detecta el fallo de red.
- Lógica de Caché Local:
 1. Tras el fallo de red, el Repository consulta la ScanHistory (Tabla 1) en SQLite buscando el url_hash_prefix.
 2. Si se encuentra y el api_validation_timestamp es reciente (ej. < 24 horas), se sirve el resultado cacheado (Seguro/Malicioso).
 3. Si no se encuentra o el caché es muy antiguo, se retorna un estado "Desconocido"
 4. La UI mostrará un mensaje claro: "No se pudo validar la URL. Proceda con precaución" (Veredicto Gris), manteniendo la funcionalidad de escaneo.

6.3 Gestión de Continuidad

La gestión de continuidad se enfoca en la persistencia de los datos del usuario (historial y configuraciones), que son locales, ante la reinstalación de la app o cambio de dispositivo.

- Copia de Seguridad Automática (Auto-Backup): La app utilizará el sistema de Auto-Backup nativo de Android. Este servicio (disponible para apps en Android 6.0+) respalda automáticamente la BD SQLite (ScanHistory, AppSettings) en la cuenta de Google Drive del usuario.
- Recuperación ante Fallos: Al reinstalar la app en un dispositivo nuevo (o el mismo) que esté logueado en la misma cuenta de Google, el sistema operativo restaurará automáticamente la BD SQLite, asegurando la continuidad del historial y configuraciones del usuario sin que este deba realizar ninguna acción.
- Exportación Manual Opcional: Adicionalmente, se incluirá una función en Gestionar Configuraciones para "Exportar Historial" (ej. a CSV) controlada por el usuario, como redundancia.

6.4 Plan de Mantención

Se define un plan de mantenimiento triple para asegurar la longevidad y seguridad del producto:

- Preventivo:
 - Actualización proactiva y semestral de librerías (Kotlin, ZXing, librerías de Jetpack) y del targetSdkVersion de Android para cumplir con los requisitos de Google Play.
 - Revisión de las guías de estilo de Material Design.
- Correctivo:
 - Corrección de bugs y fallos (crashes) reportados por los usuarios o detectados por Firebase Crashlytics
 - La meta será mantener una ratio de "usuarios libres de fallos" superior al 99.5%.
- Evolutivo:
 - Adaptación a nuevas amenazas (ej. si GSB API es deprecada o surge una mejor) y actualización a nuevas versiones de Android.
 - Incorporación de features solicitadas por la comunidad (ej. escaneo de imágenes de la galería), siempre que no comprometan la privacidad.

7 Sostenibilidad y Modelo de Negocio

7.1 Objetivo de negocio

El objetivo de negocio principal es posicionar a SecureQR como una herramienta gratuita, confiable y educativa que fomente la ciberseguridad ciudadana. A corto plazo, la métrica de éxito es la adopción y retención de usuarios. A largo plazo, su impacto radica en crear conciencia digital tangible, reducir el fraude en línea y generar oportunidades de colaboración B2B (Business-to-Business) con entidades que promueven la seguridad tecnológica (bancos, instituciones educativas, gobierno). El activo central del proyecto es la confianza, por lo que el modelo de negocio debe subsidiar la operación sin comprometer la privacidad del usuario final.

7.2 Alternativas de modelo de negocio

Modelo	Descripción	Ventajas	Desventajas
Publicidad ética	Integración de anuncios discretos no invasivos.	Integración de anuncios discretos no invasivos.	- Conflicto directo con la Propuesta de Valor. - Requiere SDKs de anuncios (vectores de rastreo). - Destruye la confianza y la promesa de "sin rastreo" - El usuario de seguridad es adverso a la publicidad.
Licenciamiento corporativo	Entrega de versión adaptada (SDK o white-label) a empresas o instituciones (ej. bancos).	- Alto alineamiento estratégico. - Ingresos B2B estables y de alto valor. - Refuerza el prestigio de la marca. - Permite que el B2B financie la app B2C gratuita.	- Requiere un equipo comercial y de soporte técnico B2B dedicado. - Ciclo de venta largo y complejo.
Financiamiento público / académico	Participación en fondos de innovación, grants de ciberseguridad o fondos gubernamentales.	- Compatible 100% con el enfoque educativo y social. - No requiere comprometer la privacidad. - Ideal para las fases iniciales (desarrollo, prototipo).	- Depende de convocatorias. - No es sostenible a largo plazo. - Depende de convocatorias y burocracia. - Los fondos suelen ser para proyectos nuevos, no para mantención.
Código abierto con soporte de pago	Publicación libre del código (FOSS) con servicios de soporte o capacitación corporativa.	- Máxima transparencia y confianza comunitaria. - Genera reputación y colaboración.	- Menor control comercial y estratégico. - Dificultad para monetizar el soporte si el código es simple. - Riesgo de clones competidores.

7.3 Estrategia de protección de datos

La estrategia de protección de datos es la piedra angular del proyecto y no es negociable. Se basa en los siguientes pilares técnicos y de cumplimiento:

- Principio de Cero-Datos (No almacenamiento PII): La aplicación está diseñada bajo el principio de Privacy by Design. No se almacenan datos personales. La arquitectura de cliente-ligero (sin backend propio) hace técnicamente imposible que el proyecto recopile PII.
- Datos Locales y Anónimos: Toda la información (URLs escaneadas) se mantiene localmente en la base de datos SQLite del dispositivo, dentro del sandbox (entorno aislado) de la aplicación. El usuario tiene control total para eliminar su historial en cualquier momento.
- Cumplimiento Regulatorio Simplificado: Al no procesar ni almacenar PII, el cumplimiento de normativas como GDPR y la ley chilena 19.628 se simplifica. El cumplimiento se basa en la minimización de datos (llevada a cero) en lugar de en la gestión de consentimientos complejos.
- Transparencia Total: La Política de Privacidad será visible y transparente dentro de la app. Esta debe declarar explícitamente: (1) que no se recopila PII, (2) que las URLs escaneadas se envían a un tercero (Google Safe Browsing API) con el único fin de validación de seguridad, y (3) que el historial es 100% local.

8 Plan de Presupuesto

La factibilidad económica es alta ya que el diseño arquitectónico de cliente-ligero elimina los costos recurrentes de hosting y servidores. El presupuesto de \$500.000 cubre exclusivamente los costos operacionales mínimos, como la cuota de registro de la aplicación en la tienda de Google y los gastos de pruebas de compatibilidad y documentación, aprovechando al máximo las herramientas gratuitas y de código abierto (Kotlin, ZXing, Google Safe Browsing API).

La cifra dada cubre los siguientes gastos:

- \$30.000 – Cuota de cuenta de desarrollador de Google Play
- \$50.000 – Gastos de impresión, encuadernación de informes y materiales de presentación.
- \$250.000 - Inversión mínima para compra de dispositivos económicos o accesorios para realizar pruebas de compatibilidad reales
- \$170.000 – Margen de seguridad para eventualidades menores

9 Ajustes del Cronograma

9.1 Revisión de Carta Gantt o desarrollo de fases

El avance con respecto a la carta Gantt, ya estamos terminando por la fase de completar el prototipo.

Anexo 1 – Carta Gantt

10 Conclusiones:

SecureQR representa una solución tecnológica y educativa efectiva frente al creciente riesgo del Quishing. Al integrar herramientas de validación en tiempo real, cifrado SSL y un enfoque libre de datos personales, la aplicación promueve un uso más seguro de los códigos QR y fortalece la confianza digital del usuario. La estructura modular y la arquitectura cliente-ligera hacen que SecureQR sea escalable, ligera y adaptable, mientras que su modelo gratuito con visión ética facilita la adopción masiva sin comprometer la privacidad. El proyecto demuestra viabilidad técnica, pertinencia social y potencial de expansión futura por ejemplo, hacia iOS, sistemas empresariales o inteligencia artificial para detección predictiva, constituyéndose en un aporte real al ámbito de la ciberseguridad móvil.

11 Referencias Bibliográficas

- Android. (s.f.). Guía de arquitectura de apps. Android Developers. Recuperado el 4 de noviembre de 2025, de <https://developer.android.com/topic/architecture>
- Android. (s.f.). Descripción general de ViewModel. Android Developers. Recuperado el 4 de noviembre de 2025, de <https://developer.android.com/topic/libraries/architecture/viewmodel>
- Google. (s.f.). Safe Browsing API (v4). Google Developers. Recuperado el 4 de noviembre de 2025, de <https://developers.google.com/safe-browsing/v4?hl=es-419>
- International Organization for Standardization. (2011). ISO/IEC 25010:2011(en) Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) System and software quality models. <https://www.iso.org/home.html>
- OWASP Foundation. (2024). OWASP Mobile Top 10 2024. owasp.org. Recuperado el 4 de noviembre de 2025, de <https://owasp.org/www-project-mobile-top-10/>

12 Anexos

Anexo	A:	Carta	Gantt	Actualizada	(Post-Prototipo)
Anexo B: Diagramas					